Policy Project: Advise the State Health Commissioner

Introduction to Statistics for Data Science and Machine Learning · Student project

1. Your role and the problem

You are a data analyst at the State Ministry of Health. Last year the Ministry ran a malaria prevention programme (bed nets plus health education) in three Local Government Areas (LGAs): A, B and C. The Commissioner must now decide one of three things:

- **Scale up** the programme to the whole state.
- **Fix first**: change how the programme is run before scaling.
- **Stop** and spend the money elsewhere.

The Commissioner is not a statistician. Your job is to analyse a survey of 600 households with Python, and then give clear advice that is backed by your results. The numbers are only useful if they lead to a decision, so the last part of the project (the policy brief) carries the most marks.

2. What you hand in

1. **One Python file or notebook** that runs from top to bottom without errors. Every line of code must have a comment in plain language.
2. **A policy brief of at most 2 pages** (Word or PDF) addressed to the Commissioner. Section 7 explains how to write it.

Your facilitator will give you the deadline and tell you how to submit.

3. The data

Your facilitator will give you the file `malaria_programme_data.csv`. Put it in the same folder as your Python code. Every student gets the same file, so your results can be compared. Each row is one household.

Column	What it means	Type
household_id	Unique number of the household	identifier
lga	LGA of the household: A, B or C	categorical
programme	Yes if the household took part in the programme, No otherwise	categorical (two groups)
uses_net	Yes if the household sleeps under a bed net	categorical (two groups)
haemoglobin	Haemoglobin level in g/dL (low levels can be linked to malaria)	continuous
clinic_visits	Number of clinic visits last month	count
days_malaria_free	Days until the next malaria episode	continuous, waiting time
distance_km	Distance to the nearest clinic in km	continuous

Treat the 600 households as a **sample**. The **population** the Commissioner cares about is all households in the state.

4. Quick reference: which test, which tail

Question	Test to use
Is one group mean different from a known value, and the population standard deviation is known?	One-sample z-test
Is one group mean different from a known value, and the standard deviation is NOT known?	One-sample t-test
Do two groups have different means?	Two-sample t-test
Are two categorical variables related?	Chi-square test of independence

Tails. Choose the tail from the question you are asking, before you look at the p-value.

- **Left-tailed**: you ask if something is LOWER than a value (H1 uses <).
- **Right-tailed**: you ask if something is HIGHER than a value (H1 uses >).

- **Two-tailed**: you ask if something is DIFFERENT, in either direction (H1 uses ≠).

Decision rule. If the p-value is smaller than α (we use α = 0.05), reject H0. Otherwise, fail to reject H0. We never say we accept H0, because failing to find evidence is not the same as proving there is none.

5. The tasks

Part A: Describe the data (20 marks)

- **A1.** For haemoglobin, clinic_visits, days_malaria_free and distance_km, report the mean, median, variance and standard deviation. Use the sample formulas (n − 1). Report the mode for clinic_visits.
- **A2.** Use the 1.5 × IQR rule to look for outliers in distance_km. Look at the largest values and decide which ones are truly extreme. Decide whether to keep or remove them, and explain your reason in two or three sentences. Report the mean, median and skewness before and after.
- **A3.** Report the skewness of days_malaria_free. Is it symmetric, right-skewed or left-skewed? What does that tell you about the shape of this variable?
- **A4.** In one short paragraph, say what the sample is, what the population is, and one reason why the sample might not perfectly represent the population.

Part B: Use probability distributions (15 marks)

- **B1. Binomial.** Using the net-use proportion in the data, find the probability that at least 15 of 20 randomly visited households use a bed net. Explain what this number means for a field team.
- **B2. Poisson.** Estimate the average number of clinic visits per household per month. Check whether the mean and the variance are close (a Poisson variable has mean equal to variance). Find the probability of more than 3 visits and the probability of no visit.
- **B3. Exponential.** Using the average days until the next malaria episode, find the probability that a household stays malaria-free for more than 90 days, and the probability of an episode within 30 days.
- **B4. Shape of haemoglobin.** Draw a histogram and report the skewness. Say in one sentence whether a normal distribution is a reasonable model.

Part C: Draw conclusions with tests and confidence intervals (30 marks)

For every test, write the following in your file as comments or text: H0, H1, the tail you chose and why, α, the test statistic, the p-value, the decision, and one sentence of interpretation in plain language.

- **C1. One-sample z-test.** The national benchmark for mean haemoglobin is 11.5 g/dL. Assume the population standard deviation is known to be 1.5 g/dL. Is mean haemoglobin in the state below the benchmark? Report a 95% confidence interval for the mean. (6 marks)
- **C2. Two-sample t-test.** Do programme households and non-programme households differ in haemoglobin? Report the difference in means with its 95% confidence interval and Cohen d. Then answer: is the result statistically significant, and is it large enough to matter in practice? (8 marks)
- **C3. Chi-square, first table.** Is bed-net use related to the LGA? Check that all expected counts are at least 5. Use the row percentages to say which LGA is the problem. (6 marks)
- **C4. Chi-square, second table.** Is bed-net use related to taking part in the programme? Run the test with and without the continuity correction, and say whether your decision changes. (4 marks)
- **C5. Confidence interval for a proportion.** Find a 95% confidence interval for the share of households using nets in the whole sample. The state target is 80%. Does the interval contain 80%? What does that say? (3 marks)
- **C6. Errors.** Pick ONE of your tests (not C4) and explain in plain words what a Type I error and a Type II error would mean for the Commissioner in this situation. (3 marks)

Part D: The policy brief (30 marks)

See Section 7.

6. Python toolbox

Every block below is complete: copy it into your own file, run it, and then change it to answer the tasks. The blocks use library functions only, so you do not need to write any function of your own. Use the output of the blocks as evidence, and write your interpretation in your own words.

You need Python with `pandas`, `numpy`, `scipy` and `matplotlib` installed. The confidence interval code in Block 5 needs SciPy 1.10 or newer.

Block 1: Load the data and describe it

```
# Import pandas, the library for working with tables of data
import pandas as pd

# Read the CSV file into a table (called a DataFrame); the file must be in
the same folder as this code
df = pd.read_csv('malaria_programme_data.csv')

# Show the number of rows and columns (each row is one household)
print(df.shape)
# Count the missing values in the whole table (0 means the data is complete)
print(df.isna().sum().sum())

# Pick out the haemoglobin column and give it a short name
hb = df['haemoglobin']
# Mean: the ordinary average
print('Mean:', hb.mean())
# Median: the middle value when the data is sorted
print('Median:', hb.median())
# Mode: the most common value; .mode() can return several values, so [0]
takes the first one
print('Mode:', hb.mode()[0])
# Sample variance; ddof=1 means divide by n-1 (see the note under this block)
print('Variance:', hb.var(ddof=1))
# Sample standard deviation, also with ddof=1
print('SD:', hb.std(ddof=1))

# Count how many households are in each LGA
print(df['lga'].value_counts())
# Compare haemoglobin between programme and non-programme households
print(df.groupby('programme')['haemoglobin'].agg(['count', 'mean', 'median',
'std']))
```

Where tools and textbooks differ. Your textbook sample variance divides by n − 1. pandas also divides by n − 1 by default, but NumPy (`np.var`, `np.std`) divides by n by default. That is why we write `ddof=1` explicitly: ddof means delta degrees of freedom, and ddof=1 turns n into n − 1. Always state which one you used.

Block 2: Outliers and skewness

```
# Import pandas for the data table
import pandas as pd
# Import scipy's statistics tools (we use it for skewness)
from scipy import stats

# Read the data file into a table
df = pd.read_csv('malaria_programme_data.csv')

# Pick out the distance column and give it a short name
d = df['distance_km']
# First quartile Q1: the value that 25% of households fall below
q1 = d.quantile(0.25)
# Third quartile Q3: the value that 75% of households fall below
q3 = d.quantile(0.75)
# Interquartile range: the spread of the middle 50% of the data
iqr = q3 - q1
# Lower fence: anything below this is flagged as an outlier
lower_fence = q1 - 1.5 * iqr
# Upper fence: anything above this is flagged as an outlier
upper_fence = q3 + 1.5 * iqr
# Print the fences so you can see where the cut-offs are
print('Fences:', lower_fence, upper_fence)

# Keep only the rows that fall outside the fences
outliers = df[(d < lower_fence) | (d > upper_fence)]
# Count how many households were flagged
print('Number flagged:', len(outliers))
# Show the 6 largest distances so you can judge which ones are truly extreme
print(df.sort_values('distance_km', ascending=False).head(6))

# Skewness using pandas (it uses the adjusted formula)
print('Skewness (pandas):', d.skew())
# Skewness using scipy's default (it uses the plain formula, so the number
differs slightly)
print('Skewness (scipy default):', stats.skew(d))
# Mean and median side by side: a mean far above the median points to right
skew
print('Mean:', d.mean(), 'Median:', d.median())

# Make a second version of the data with the extreme distances (over 30 km)
removed
```

```
d_clean = d[d <= 30]
# Recompute the same summaries so you can compare before and after
print('After removal -> Mean:', d_clean.mean(), 'Median:', d_clean.median(),
'Skewness:', d_clean.skew())
```

Where tools and textbooks differ. Quartiles can be computed in more than one valid way. pandas and NumPy use linear interpolation at position p × (n − 1) (counting from 0), while some textbooks use position p × (n + 1) (counting from 1). With 600 values the two methods give almost the same answer, but with a small sample they can differ, so always say which method you used. Skewness also has two common formulas: pandas gives the adjusted version, while `scipy.stats.skew` gives the plain version by default (add `bias=False` to match pandas). The 1.5 × IQR rule only flags unusual values. It does not tell you to delete them: you must decide that from what the values mean in real life.

Block 3: Binomial, Poisson and exponential probabilities

```
# Import pandas for the data table
import pandas as pd
# Import scipy's statistics tools: it holds the binomial, Poisson and
exponential distributions
from scipy import stats
# Import matplotlib to draw a histogram
import matplotlib.pyplot as plt

# Read the data file into a table
df = pd.read_csv('malaria_programme_data.csv')

# --- Binomial: number of net users among 20 visited households ---
# Observed share of households that use a net (True counts as 1, so the mean
is the proportion)
p_net = (df['uses_net'] == 'Yes').mean()
# P(at least 15 of 20) = P(X > 14); sf means 'survival function' = 1 minus
the cumulative probability
print('P(at least 15 of 20 use nets):', stats.binom.sf(14, n=20, p=p_net))

# --- Poisson: number of clinic visits in a month ---
# The Poisson average rate (lambda) is estimated by the sample mean
lam = df['clinic_visits'].mean()
# A Poisson variable has mean equal to variance, so compare the two as a
quick check
print('Mean:', lam, 'Variance:', df['clinic_visits'].var(ddof=1))
# P(more than 3 visits) = P(X > 3)
```

```
print('P(more than 3 visits):', stats.poisson.sf(3, mu=lam))
# P(exactly 0 visits); pmf gives the probability of one exact value
print('P(0 visits):', stats.poisson.pmf(0, mu=lam))

# --- Exponential: days until the next malaria episode ---
# The exponential average waiting time is estimated by the sample mean
mean_days = df['days_malaria_free'].mean()
# scipy uses scale = mean waiting time (which is 1 divided by the rate in the
textbook formula)
print('P(malaria-free for more than 90 days):', stats.expon.sf(90,
scale=mean_days))
# P(episode within 30 days) = cumulative probability up to 30
print('P(episode within 30 days):', stats.expon.cdf(30, scale=mean_days))

# --- Shape check for haemoglobin (is it roughly normal?) ---
# Skewness near 0 suggests a symmetric, bell-like shape
print('Haemoglobin skewness:', df['haemoglobin'].skew())
# Draw a histogram with 20 bars
plt.hist(df['haemoglobin'], bins=20, edgecolor='black')
# Label the horizontal axis with the unit
plt.xlabel('Haemoglobin (g/dL)')
# Label the vertical axis
plt.ylabel('Number of households')
# Show the picture on screen
plt.show()
```

Where tools and textbooks differ. Textbooks write the exponential distribution with a rate λ, and the mean waiting time is 1/λ. SciPy does not use the rate: it uses `scale`, which IS the mean waiting time. If you pass the rate as the scale, your answer will be wrong. Also note that this simple approach treats every household as if it was followed until its episode, and it uses a single average for everyone. That is a simplification you can mention in your limits.

Block 4: One-sample z-test and confidence interval

```
# Import numpy for the square root
import numpy as np
# Import pandas for the data table
import pandas as pd
# Import scipy's statistics tools: the normal distribution gives us z-test
probabilities
from scipy import stats
```

```
# Read the data file into a table
df = pd.read_csv('malaria_programme_data.csv')

# Pick out the haemoglobin column
x = df['haemoglobin']
# The national benchmark we compare against (the value in H0)
mu0 = 11.5
# The population standard deviation, which this test assumes is known
sigma = 1.5
# Sample size
n = len(x)
# Sample mean
xbar = x.mean()

# Standard error: how much a sample mean typically wobbles from sample to
sample
se = sigma / np.sqrt(n)
# z statistic: how many standard errors the sample mean is from the benchmark
z = (xbar - mu0) / se
print('Sample mean:', xbar, 'SE:', se, 'z:', z)

# Left-tailed p-value: the area under the normal curve to the LEFT of z
p_left = stats.norm.cdf(z)
# Right-tailed p-value: the area to the RIGHT of z (survival function)
p_right = stats.norm.sf(z)
# Two-tailed p-value: the area in both tails, so double the smaller tail
p_two = 2 * stats.norm.sf(abs(z))
print('Left:', p_left, 'Right:', p_right, 'Two-tailed:', p_two)

# Critical z value for a 95% confidence interval (2.5% in each tail)
z_crit = stats.norm.ppf(0.975)
# Margin of error: critical value multiplied by the standard error
margin = z_crit * se
# 95% confidence interval for the true mean haemoglobin
print('95% CI:', xbar - margin, xbar + margin)
```

Reading the output. The standard error (SE) tells you how precise the sample mean is: a smaller SE means a more precise estimate. The margin of error is the critical value times the SE, and the confidence interval is the sample mean plus and minus the margin of error. Only ONE of the three p-values answers your question: pick the one that matches your H1.

Where tools and textbooks differ. SciPy has no ready-made one-sample z-test, so we use the formula and the normal distribution directly. The `statsmodels` library has a `ztest` function if you want to check your answer. In real life the population standard deviation is rarely known, and then a t-test is the honest choice. Here we are told to treat it as known so you can practise the z-test.

Block 5: Two-sample t-test, effect size and confidence interval

```
# Import numpy for the square root
import numpy as np
# Import pandas for the data table
import pandas as pd
# Import scipy's statistics tools, which include the t-test
from scipy import stats

# Read the data file into a table
df = pd.read_csv('malaria_programme_data.csv')

# Haemoglobin values for programme households
yes = df.loc[df['programme'] == 'Yes', 'haemoglobin']
# Haemoglobin values for non-programme households
no = df.loc[df['programme'] == 'No', 'haemoglobin']

# Welch two-sample t-test (does not assume equal spread in the two groups);
two-tailed by default
res = stats.ttest_ind(yes, no, equal_var=False)
# Print the t statistic, the p-value and the degrees of freedom
print('Two-tailed Welch:', res.statistic, res.pvalue, res.df)

# The same test, but only asking 'is the programme mean HIGHER?' (right-
tailed)
res_right = stats.ttest_ind(yes, no, equal_var=False, alternative='greater')
# Print the right-tailed result
print('Right-tailed Welch:', res_right.statistic, res_right.pvalue)

# The classic textbook version that assumes equal spread (pooled variance),
for comparison
res_pooled = stats.ttest_ind(yes, no, equal_var=True)
# Print it; the answer is very close here because both groups have similar
spread
print('Pooled (Student):', res_pooled.statistic, res_pooled.pvalue)
```

```
# Difference between the two group means (this is the EFFECT SIZE in real
units, g/dL)
diff = yes.mean() - no.mean()
# 95% confidence interval for that difference (needs SciPy 1.10 or newer)
ci = res.confidence_interval(confidence_level=0.95)
# Print the difference and its interval
print('Difference:', diff, '95% CI:', ci.low, ci.high)

# Cohen d: the difference measured in standard deviations (about 0.2 = small,
0.5 = medium, 0.8 = large)
d = diff / np.sqrt((yes.var(ddof=1) + no.var(ddof=1)) / 2)
# Print it to judge whether the difference matters in practice
print('Cohen d:', d)
```

Where tools and textbooks differ. Many textbooks teach the pooled t-test, which assumes both groups have the same variance. SciPy runs the pooled version by default, but the Welch version (`equal_var=False`) is safer because it works well whether or not the variances are equal, so we recommend it. Also, SciPy gives a two-tailed p-value unless you set `alternative`. For a right-tailed test the p-value is half of the two-tailed one ONLY when the difference is in the direction of H1.

Statistical versus practical significance. A p-value below 0.05 says the difference is unlikely to be zero. It does not say the difference is big. Always look at the size of the difference, its confidence interval and Cohen d, and ask whether the difference is large enough to justify the cost of a state-wide programme.

Block 6: Chi-square tests of independence

```
# Import pandas for the data table and the cross-tabulation
import pandas as pd
# Import scipy's statistics tools, which include the chi-square test
from scipy import stats

# Read the data file into a table
df = pd.read_csv('malaria_programme_data.csv')

# Build the table of counts: rows = LGA, columns = uses net (No/Yes)
tab = pd.crosstab(df['lga'], df['uses_net'])
# Print the observed counts
print(tab)

# Run the chi-square test of independence; it returns 4 things
```

```
chi2, p, dof, expected = stats.chi2_contingency(tab)
# Print the test statistic, p-value and degrees of freedom
print('Chi-square:', chi2, 'p-value:', p, 'df:', dof)

# Turn the expected counts into a table with the same labels, rounded for
reading
print(pd.DataFrame(expected, index=tab.index, columns=tab.columns).round(1))
# Check the rule of thumb: every expected count should be at least 5
print('All expected counts >= 5:', (expected >= 5).all())

# Row percentages show WHERE the difference is (share of Yes/No inside each
LGA)
print(pd.crosstab(df['lga'], df['uses_net'], normalize='index').round(3))

# Second table: programme (rows) versus net use (columns), a 2 x 2 table
tab2 = pd.crosstab(df['programme'], df['uses_net'])
# Print the observed counts
print(tab2)
# scipy applies Yates' continuity correction to 2 x 2 tables by default
print('With Yates correction:', stats.chi2_contingency(tab2, correction=True)
[:3])
# Many textbooks use no correction, so switch it off to compare
print('Without correction:', stats.chi2_contingency(tab2, correction=False)
[:3])
```

Reading the output. H0 says the two variables are independent (no relationship). The test compares the observed counts with the counts you would expect if H0 were true. A large chi-square value, and therefore a small p-value, means the observed counts are too far from the expected ones to be explained by chance. The test tells you THAT there is a relationship, not WHERE it is, so use the row percentages to see where.

Where tools and textbooks differ. For a 2 × 2 table SciPy applies Yates′ continuity correction by default, which makes the chi-square value a little smaller. Many textbooks leave it out. Both are accepted, so state which one you used. The correction is only applied to tables with 1 degree of freedom, so it does not affect the 3 × 2 table.

Block 7: Confidence interval for a proportion

```
# Import numpy for the square root
import numpy as np
# Import pandas for the data table
import pandas as pd
```

```
# Import scipy's statistics tools: the normal distribution gives the critical
value
from scipy import stats

# Read the data file into a table
df = pd.read_csv('malaria_programme_data.csv')

# True for every household that uses a net, False otherwise
used = df['uses_net'] == 'Yes'
# Sample proportion: the share of households using nets
p_hat = used.mean()
# Sample size
n = len(used)
# Standard error of a proportion
se = np.sqrt(p_hat * (1 - p_hat) / n)
# Critical z value for 95% confidence
z_crit = stats.norm.ppf(0.975)
# Lower and upper ends of the 95% confidence interval
low = p_hat - z_crit * se
high = p_hat + z_crit * se
# Print the estimate and its interval
print('Proportion:', p_hat, '95% CI:', low, high)
# Check whether the 80% coverage target lies inside the interval
print('Is 0.80 inside the interval?', low <= 0.80 <= high)
```

Where tools and textbooks differ. This is the simple (Wald) interval taught in most introductory courses. Some software uses other methods, such as the Wilson interval, which behave better when the sample is small or the proportion is close to 0 or 1. With 600 households and a proportion near one half, the difference is tiny.

7. Writing the policy brief (Part D, 30 marks)

The Commissioner will read your brief in five minutes. Write in plain language and do not paste code or raw Python output. Use these four sections:

1. **Recommendation (10 marks).** One clear sentence first: scale up, fix first, or stop. Then two or three sentences on why. If you recommend fixing first, say exactly WHAT to fix and WHERE.
2. **Evidence (8 marks).** Three or four key results, each in plain language with the number that supports it. Use confidence intervals, not only p-values. Link every result to your recommendation.

3. **Risk of being wrong (6 marks).** Explain what a Type I error and a Type II error would cost the state in this decision, and which one worries you more. Say how that affects how cautious your recommendation is.
4. **Limits (6 marks).** At least three honest limits of your analysis. Think about sample size, whether households were chosen fairly, the difference between association and cause (did the programme cause the change, or were the households different to begin with?), and statistical versus practical significance.

Example of weak and strong writing.

- Weak: p = 0.03, so we reject H0.
- Strong: Households in the programme scored X units higher on average (95% confidence interval: a to b). The difference is unlikely to be due to chance, but the lower end of the interval is small, so we should be careful before assuming a large benefit.

Two students can reach different recommendations from the same data. That is fine, as long as the evidence you give really supports your choice.

8. Marking guide (100 marks)

Section	Marks	What earns the marks
Part A: Describe the data	20	Correct summaries with the right formulas; a sensible and justified decision about outliers; correct reading of skewness; clear sample versus population statement
Part B: Distributions	15	Right distribution for each variable; correct probabilities; each number explained in words
Part C: Tests and confidence intervals	30	For each test: correct hypotheses and tail, correct test choice, correct decision, plain-language interpretation. For the t-test, separate statistical from practical significance
Part D: Policy brief	30	Clear recommendation (10); evidence in plain language (8); Type I and Type II reasoning (6); honest limits (6)

Section	Marks	What earns the marks
Code quality	5	Runs from top to bottom; every line commented; clear names
Total	**100**	

9. Rules

- Use library functions (`pandas`, `numpy`, `scipy`, `matplotlib`). You do not need to write your own functions.
- The code blocks in this document are there to help you. Your comments, your interpretation and your brief must be in your own words.
- Show the output next to every interpretation, so the marker can check your numbers.
- You may discuss ideas with classmates, but each student submits their own code and their own brief. Briefs that are copied from another student will receive zero.
- If a result surprises you, check your code and your hypotheses first. Then report what you found honestly, even if it is not what you expected.